



数据库工程作业

宠物信息管理系统

专	业：	计算机科学与技术
学	号：	2413633
班	号：	0999
姓	名：	焦宇堃

2026 年 6 月 7 日

目录

1	项目概况 (10 分)	1
1.1	基本信息	1
1.2	系统主要功能简介 (4 分)	1
1.3	系统主要页面截图 (6 分)	1
2	系统配置 (10 分)	2
2.1	评分点对应	2
2.2	配置环境说明 (2 分)	2
2.3	数据库连接方式与连接参数分析 (8 分)	2
3	数据库设计 (14 分)	3
3.1	评分点对应	3
3.2	总体设计思路	3
3.3	数据表创建顺序与参照关系 (10 分)	4
3.4	代表性设计说明	4
3.4.1	主人、宠物与猫狗子表	4
3.4.2	库存表的复合主键设计	4
3.4.3	计划与日志对象	4
3.5	关系图 (4 分)	5
4	含有事务应用的删除操作 (13 分)	5
4.1	评分点对应	5
4.2	选取的代表场景	5
4.3	功能描述 (1 分)	5
4.4	涉及的表、连接字段与删除条件 (4 分)	6
4.5	代表事件 A: 狗类宠物改为猫类宠物	6
4.6	代表事件 B: 猫类宠物改为狗类宠物	6
4.7	事务边界与关键代码 (4 分)	6
4.8	程序演示说明 (4 分)	8
5	触发器控制下的添加操作 (20 分)	8
5.1	评分点对应	8
5.2	选取的代表场景	8
5.3	功能描述 (1 分)	9
5.4	触发器功能描述 (2 分)	9
5.5	涉及的表与输入数据规则 (3 分)	9
5.6	代表事件 A: 合法插入	9
5.7	代表事件 B: 非法插入	9
5.8	关键 SQL 与程序演示 (14 分)	9
6	存储过程控制下的更新操作 (18 分)	11
6.1	评分点对应	11
6.2	选取的代表场景	11
6.3	功能描述 (1 分)	11

6.4	存储过程功能描述（1 分）	11
6.5	涉及的关系表、连接字段与更改规则（5 分）	11
6.6	代表事件 A：自动选择最早过期批次并扣减库存	11
6.7	代表事件 B：指定批次不存在	12
6.8	代表事件 C：库存不足	12
6.9	关键代码与程序演示（11 分）	12
7	含有视图的查询操作（15 分）	14
7.1	评分点对应	14
7.2	视图总体设计	14
7.3	代表查询 A：查看某个主人的全部宠物	15
7.4	代表查询 B：查看药物库存及即将过期记录	15
7.5	代表查询 C：查看某只宠物或某个主人的喂药计划	15
7.6	创建视图与后端查询代码（6 分）	16
7.7	程序演示（4 分）	17
8	总结	18

1 项目概况（10 分）

1.1 基本信息

本项目名称为“宠物信息管理系统”，面向宠物主人场景组织主人、宠物、食品库存、药品库存与喂药计划等核心数据。

1.2 系统主要功能简介（4 分）

系统完成的核心功能可以概括为以下五类：

- 1. 账号登录与本人信息查看。用户通过主人编号和密码登录，登录后只能访问与自己相关的数据。
- 2. 宠物信息管理。系统可以查看主人名下全部宠物，并支持修改宠物基础信息；当宠物类别在“猫”和“狗”之间切换时，后端会同步维护子表数据。
- 3. 食品与药品库存管理。系统记录不同批次的库存数量、过期时间和厂商信息，支持查询、录入和更新。
- 4. 喂药计划管理。系统可以查看不同宠物的喂药计划，并通过存储过程执行计划、减少对应药品批次库存。
- 5. 操作日志与约束控制。数据库层利用触发器、存储过程和日志过程记录关键操作，并把核心业务约束尽量下沉到数据库内部。

1.3 系统主要页面截图（6 分）

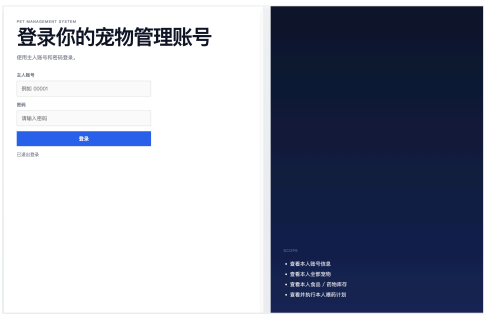


图 1 登录页



图 2 主人信息与宠物列表页

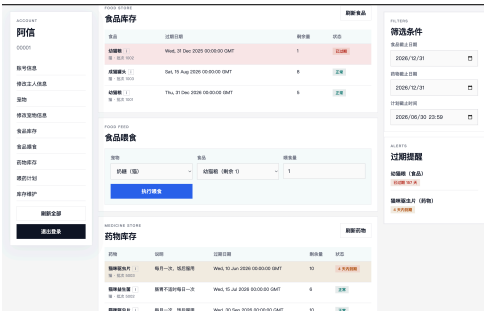


图 3 食物 & 药物页



图 4 喂药计划页

2 系统配置（10 分）

2.1 评分点对应

本节对应作业要求中的两部分内容：一是说明系统配置情况，占 2 分；二是说明如何通过连接串或连接参数让高级语言连接数据库，并分析连接项的作用，占 8 分。

2.2 配置环境说明（2 分）

在我的大作业中，后台数据库采用 MySQL，应用层使用 Python 语言，Web 框架为 Flask，数据库驱动为 PyMySQL，通过浏览器访问 Flask 提供的接口和静态页面，属于 B/S 架构。为了保证运行环境独立，项目先在本地创建虚拟环境，再安装依赖并启动应用。具体步骤如下：

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
4 python3 -m py_compile app.py db.py config.py
5 DB_PASSWORD='password' DB_NAME='pet' python3 app.py
```

Listing 1: Python 后端环境配置与启动命令

2.3 数据库连接方式与连接参数分析（8 分）

应用层没有把连接参数直接写死在业务代码中，而是由 config.py 统一读取环境变量，再由 db.py 中的 create_connection() 调用 PyMySQL 建立连接。这样做的好处是切换数据库账号、端口和密码时不需要修改业务逻辑。

序号	名称	功能说明	默认值
1	host	指定数据库服务器地址，决定当前应用要连接哪一台 MySQL 主机。	127.0.0.1
2	port	指定数据库监听端口，用于连接本地 MySQL 服务。	3306
3	user	指定连接数据库所使用的账号。	root
4	password	指定数据库账号口令，用于身份认证。	环境变量 DB_PASSWORD
5	database	指定连接成功后默认进入的数据库。	pet
6	charset	指定字符集，保证中文昵称、地址和说明等内容能够正确存取。	utf8mb4
7	autocommit	关闭自动提交，由程序在关键操作后显式执行 commit 或 rollback。	False

连接代码如下所示：

```
1 def create_connection():
2     return pymysql.connect(
3         host=Config.DB_HOST,
4         port=Config.DB_PORT,
5         user=Config.DB_USER,
6         password=Config.DB_PASSWORD,
7         database=Config.DB_NAME,
8         charset="utf8mb4",
9         cursorclass=DictCursor,
10        autocommit=False,
11    )
```

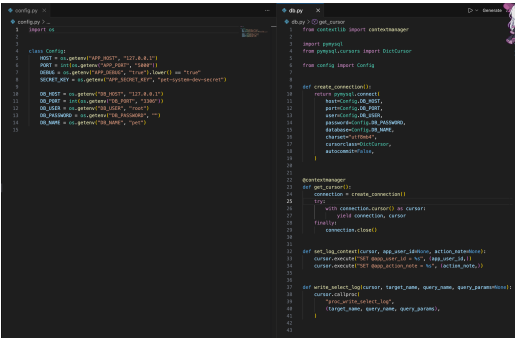


图 5 config.py 与 db.py 中的数据库连接代码截图

3 数据库设计（14 分）

3.1 评分点对应

本节对应作业要求中的两部分内容：一是按建表顺序给出数据表、主键、参照属性和被参照属性，占 10 分；二是给出关系图，占 4 分。为了更完整地体现本项目工作量，除列出全部业务表外，还把辅助日志表一并纳入说明。

3.2 总体设计思路

数据库的核心功能如下：

- 1. owner_table 记录主人信息；
- 2. pet 记录宠物主表信息，cat 和 dog 分别补充两类宠物的特有属性；
- 3. food、medicine 与各自的库存表负责管理用品和药品；
- 4. medicine_plan 记录喂药计划，是触发器、视图和存储过程的重要交汇点；
- 5. operation_log 负责记录增删改查日志，便于演示与追踪。

3.3 数据表创建顺序与参照关系（10 分）

顺序	数据表名称	主键	参照属性	被参照表及属性
1	owner_table	owner_id	无	无
2	food_manu	food_manu_id	无	无
3	food	food_id	(food_manu_id)	food_manu(food_manu_id)
4	food_store	(owner_id, food_id, food_store_batch_no)	(owner_id), (food_id)	owner_table(owner_id); food(food_id)
5	medicine_manu	medicine_manu_id	无	无
6	medicine	medicine_id	(medicine_manu_id)	medicine_manu(medicine_manu_id)
7	medicine_store	(owner_id, medicine_id, medicine_batch_no)	(owner_id), (medicine_id)	owner_table(owner_id); medicine(medicine_id)
8	pet	pet_id	(owner_id)	owner_table(owner_id)
9	medicine_plan	plan_id	(medicine_id), (pet_id)	medicine(medicine_id); pet(pet_id)
10	cat	pet_id	(pet_id)	pet(pet_id)
11	dog	pet_id	(pet_id)	pet(pet_id)
12	operation_log	log_id	无	无

3.4 代表性设计说明

3.4.1 主人、宠物与猫狗子表

owner_table 是整个系统的上游主实体，用来保存主人的基础信息，例如编号、昵称、联系方式、地址、生日和登录密码等内容。pet 则作为宠物主表，保存宠物编号、所属主人编号、姓名、生日、性别和类别等通用属性，并通过 owner_id 与 owner_table 建立一对多关系，表示一个主人可以拥有多只宠物。cat 和 dog 作为宠物子表，分别补充剪爪周期、常用猫砂、遛狗需求等级、犬证号等专有字段。这样设计的好处是：一方面保留了统一的宠物主表，便于按宠物这一对象统一查询；另一方面避免把猫狗特有字段全部堆在同一张表中，减少空值字段并保持结构清晰。

3.4.2 库存表的复合主键设计

food_store 和 medicine_store 都采用“主人编号 + 用品编号 + 批次号”的复合主键。这样可以在同一主人、同一商品下记录多个批次，从而支持过期时间筛选、先进先出和批次级库存扣减等后续逻辑。

3.4.3 计划与日志对象

medicine_plan 同时连接宠物和药品，是触发器、视图和存储过程集中作用的一张表。operation_log 则服务于所有增删改查操作，用于记录数据库层和应用层行为。

3.5 关系图（4 分）

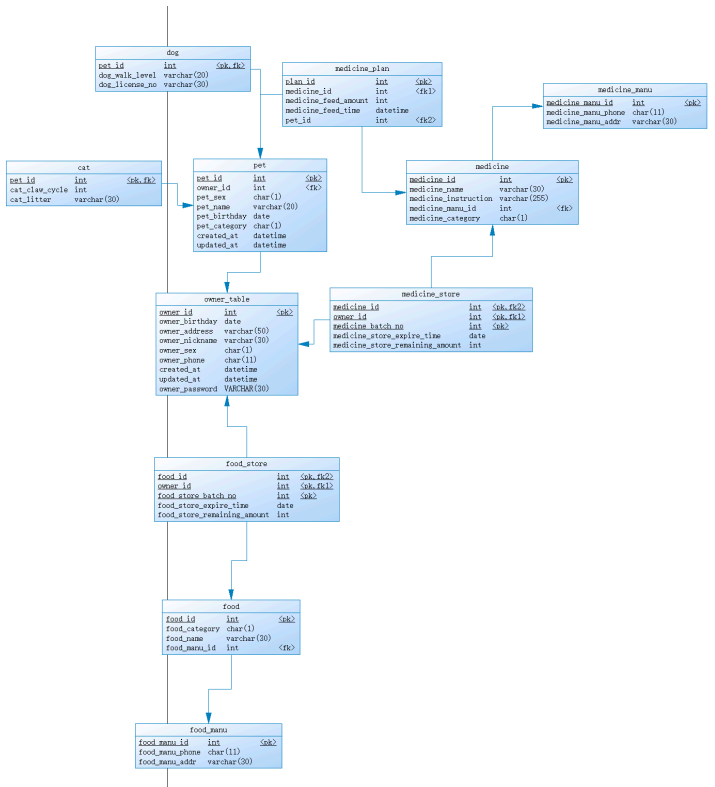


图 6 数据库关系图截图

4 含有事务应用的删除操作（13 分）

4.1 评分点对应

本节按要求覆盖以下内容：功能描述 1 分，涉及的关系表 2 分，连接字段 1 分，删除条件字段 1 分，关键代码 4 分，程序演示 4 分。为了体现工作量，本节不只写一个删除瞬间，而是选取两个类别切换场景，说明删除逻辑如何在事务中与更新、插入共同完成。

4.2 选取的代表场景

本系统中最能体现删除操作与事务配合的地方是在宠物类别变更逻辑中。用户修改宠物类别时，如果原来是狗、现在改成猫，则需要删除 dog 子表中的旧记录并写入 cat 子表；反之亦然。这里的删除不是孤立行为，而是为了保证主表 pet 与子表结构保持一致，因此适合作为事务删除的代表事件。

4.3 功能描述（1 分）

该操作所要完成的功能是：当用户修改某只宠物的类别时，系统在一次事务中同步更新宠物主表，并删除旧类别子表中的无效记录，再插入新类别子表记录，从而保证同一只宠物不会同时出现在 cat 和 dog 两张子表中。

4.4 涉及的表、连接字段与删除条件（4 分）

涉及的关系表：本操作直接涉及 pet、cat、dog 三张表。其中 pet 是宠物主表，负责保存宠物的类别与归属主人；cat 和 dog 是两张子表，分别保存猫类和狗类宠物的专有属性。删除动作虽然发生在子表上，但删除依据来自主表中的宠物编号、主人编号以及变更后的宠物类别。

表连接字段：

1. pet.pet_id = cat.pet_id, 用于把猫类扩展信息与宠物主表记录关联起来。
2. pet.pet_id = dog.pet_id, 用于把狗类扩展信息与宠物主表记录关联起来。

删除条件字段描述：该功能在执行删除前，必须先确认当前登录主人是否拥有这只宠物，随后再根据类别变化决定删除哪一张子表中的旧记录，因此可以归纳为以下三类条件：

1. pet.pet_id = ? AND pet.owner_id = ?：先定位当前主人名下要修改的目标宠物，防止越权修改。
2. dog.pet_id = ?：当原宠物类别为狗、修改后类别为猫时，删除 dog 表中的旧子类记录。
3. cat.pet_id = ?：当原宠物类别为猫、修改后类别为狗时，删除 cat 表中的旧子类记录。

4.5 代表事件 A：狗类宠物改为猫类宠物

当某只宠物原来属于狗类，而用户把类别改成猫类时，后端先更新 pet 主表中的 pet_category，随后删除 dog 表中的旧记录，再补写 cat 表中的新记录。其核心删除语句如下：

```
DELETE FROM dog WHERE pet_id = %s
```

4.6 代表事件 B：猫类宠物改为狗类宠物

反向切换时，逻辑完全对称：先更新 pet 主表，再删除 cat 表中的旧记录，并插入 dog 表。对应的删除语句为：

```
DELETE FROM cat WHERE pet_id = %s
```

4.7 事务边界与关键代码（4 分）

应用层连接使用 autocommit=False，因此整个修改逻辑运行在显式事务下。后端用 try / except 包裹宠物更新、子表删除与子表新增；只有三步都成功时才执行 commit，如果任意一步报错则执行 rollback。

```
1 old_category = pet_row["pet_category"]
2 new_category = payload.get("pet_category") or old_category
3
4 cursor.execute(
5     """
6     UPDATE pet
```

```

7     SET pet_name = COALESCE(%s, pet_name),
8         pet_sex = %s,
9         pet_birthday = COALESCE(%s, pet_birthday),
10        pet_category = %s
11    WHERE pet_id = %s
12        AND owner_id = %s
13    """
14    (
15        payload.get("pet_name"),
16        payload.get("pet_sex"),
17        payload.get("pet_birthday"),
18        new_category,
19        pet_id,
20        owner_id,
21    ),
22 )
23
24 if new_category != old_category:
25     if new_category == "猫":
26         cursor.execute("DELETE FROM dog WHERE pet_id = %s", (pet_id,))
27         cursor.execute(
28             """
29             INSERT INTO cat(pet_id, cat_claw_cycle, cat_litter)
30             VALUES(%s, %s, %s)
31             ON DUPLICATE KEY UPDATE
32                 cat_claw_cycle = VALUES(cat_claw_cycle),
33                 cat_litter = VALUES(cat_litter)
34             """,
35             (pet_id, payload.get("cat_claw_cycle") or 14, payload.get(
36 "cat_litter")),
37         )
38     else:
39         cursor.execute("DELETE FROM cat WHERE pet_id = %s", (pet_id,))
40         cursor.execute(
41             """
42             INSERT INTO dog(pet_id, dog_walk_level, dog_license_no)
43             VALUES(%s, %s, %s)
44             ON DUPLICATE KEY UPDATE
45                 dog_walk_level = VALUES(dog_walk_level),
46                 dog_license_no = VALUES(dog_license_no)
47             """,
48             (pet_id, payload.get("dog_walk_level"), payload.get("
49 dog_license_no")),
50         )

```

Listing 2: app.py 中宠物类别切换的事务性删除与补写逻辑

PET EDIT

修改宠物信息

选择宠物

奶糖 (猫)

宠物姓名

奶糖

性别

女

出生日期

2023/05/01

种类

猫

剪爪周期

14

常用猫砂

豆腐猫砂

保存宠物信息

图 7 宠物类别切换相关的应用层代码截图

```
if new_category != old_category:
    if new_category == "猫":
        cursor.execute("DELETE FROM dog WHERE pet_id = %s", (pet_id,))
        cursor.execute(
            """
            INSERT INTO cat(pet_id, cat_claw_cycle, cat_litter)
            VALUES(%s, %s, %s)
            ON DUPLICATE KEY UPDATE
            cat_claw_cycle = VALUES(cat_claw_cycle),
            cat_litter = VALUES(cat_litter)
            """
            (pet_id, payload.get("cat_claw_cycle") or 14, payload.get("cat_litter")),
        )
    else:
        cursor.execute("DELETE FROM cat WHERE pet_id = %s", (pet_id,))
        cursor.execute(
            """
            INSERT INTO dog(pet_id, dog_walk_level, dog_license_no)
            VALUES(%s, %s, %s)
            ON DUPLICATE KEY UPDATE
            dog_walk_level = VALUES(dog_walk_level),
            dog_license_no = VALUES(dog_license_no)
            """
            (pet_id, payload.get("dog_walk_level"), payload.get("dog_license_no")),
        )
```

图 8 删除旧子表记录和插入新子表记录的 SQL 截图

4.8 程序演示说明（4 分）

奶糖

猫 · 女

生日: Mon, 01 May 2023 00:00:00 GMT

剪爪周期: 14 天 · 猫砂: 豆腐猫砂

查看计划

图 9 宠物类别切换前的界面截图

pet_id	cat_claw_cycle	cat_litter	pet_id	owner_id	pet_sex	pet_name	pet_birthday	pet_category	created_at	updated_at
1	14	豆腐猫砂	1	1	女	奶糖	2023-05-01	猫	2023-05-01 10:41:44	2023-05-01 10:41:44

图 10 宠物类别切换前的数据库记录截图

奶糖

狗 · 女

生日: Mon, 01 May 2023 00:00:00 GMT

遛狗等级: - · 犬证号:

查看计划

图 11 宠物类别切换后的界面截图

pet_id	dog_walk_level	dog_license_no	pet_id	owner_id	pet_sex	pet_name	pet_birthday	pet_category	created_at	updated_at
1			1	1	女	奶糖	2023-05-01	狗	2023-05-01 10:41:44	2023-05-01 10:54:24
2		D000000001	2	2	男	旺财	2023-05-01	狗	2023-05-01 10:41:44	2023-05-01 10:54:24
3		D000000002	3	3	女	小花	2023-05-01	狗	2023-05-01 10:41:44	2023-05-01 10:54:24

图 12 宠物类别切换后的数据库记录截图

5 触发器控制下的添加操作（20 分）

5.1 评分点对应

本节按要求覆盖功能描述、触发器描述、涉及表、输入数据规则、关键代码以及程序演示。为了体现工作量，本节选择代表事件：一个合法插入、一个类型不匹配的非插入，用来展示触发器的边界。

5.2 选取的代表场景

本系统把新增喂药计划作为触发器控制下的添加操作。对应数据表为 medicine_plan，数据库中定义了 trigger_medicine_plan_check_insert。该触发器在插入前检查宠物类别与药物适用类别是否一致，不一致时直接拒绝插入。

5.3 功能描述（1 分）

该操作所要完成的功能是：向 `medicine_plan` 表中新增一条喂药计划，并在插入阶段由数据库自动验证“这只宠物能否使用该药物”。

5.4 触发器功能描述（2 分）

该触发器在 `before insert` 阶段分别查询新计划中对应宠物的 `pet_category` 和药物的 `medicine_category`。如果宠物不存在、药物不存在，或者两者类别不匹配，触发器会通过 `signal sqlstate '45000'` 主动抛出错误并终止插入。

5.5 涉及的表与输入数据规则（3 分）

涉及的表：`medicine_plan`、`pet`、`medicine`

字段	规则
<code>plan_id</code>	主键，不能与已有计划编号重复。
<code>medicine_id</code>	必须在 <code>medicine</code> 表中存在。
<code>pet_id</code>	必须在 <code>pet</code> 表中存在。
<code>medicine_feed_amount</code>	必须大于 0。
<code>medicine_feed_time</code>	不能为空，表示计划执行时间。
<code>medicine_category</code> 与 <code>pet_category</code>	两者必须一致，例如猫药只能给猫使用，狗药只能给狗使用。

5.6 代表事件 A：合法插入

合法插入样例如下：

```
1 INSERT INTO medicine_plan(plan_id, medicine_id, medicine_feed_amount,
2                             medicine_feed_time, pet_id)
3 VALUES(10, 1, 1, '2026-06-06 08:00:00', 1);
```

其中 `pet_id = 1` 对应猫类宠物，`medicine_id = 1` 对应猫类药物，因此触发器允许插入成功。

5.7 代表事件 B：非法插入

故意违反规则的插入样例：

```
1 INSERT INTO medicine_plan(plan_id, medicine_id, medicine_feed_amount,
2                             medicine_feed_time, pet_id)
3 VALUES(11, 3, 1, '2026-06-06 09:00:00', 1);
```

此时 `pet_id = 1` 仍然是猫，但 `medicine_id = 3` 对应狗药，因此触发器会报出“药物适用动物与宠物种类不匹配，不能添加喂药计划”。

5.8 关键 SQL 与程序演示（14 分）

```
1 create trigger trigger_medicine_plan_check_insert
2 before insert on medicine_plan
3 for each row
```

```
4 begin
5     declare v_pet_category char(1);
6     declare v_medicine_category char(1);
7
8     select pet_category into v_pet_category
9     from pet
10    where pet_id = new.pet_id;
11
12    select medicine_category into v_medicine_category
13    from medicine
14    where medicine_id = new.medicine_id;
15
16    if v_pet_category <> v_medicine_category then
17        signal sqlstate '45000'
18        set message_text = '药物适用动物与宠物种类不匹配，不能添加喂药计划';
19    end if;
20 end
```

Listing 3: 喂药计划插入触发器的核心校验逻辑



图 13 触发器源码截图

pet_id	owner_id	pet_sex	pet_name	pet_birthday	pet_category	created_at	updated_at	plan_id	medicine_id	medicine_feed_amount	medicine_feed_time	pet_id()
1	1	女	奶糖	2023-05-01	猫	2026-06-07 01:09:21	2026-06-07 01:09:21	1	1	1	2026-06-05 08:00:00	1
1	1	女	奶糖	2023-05-01	猫	2026-06-07 01:09:21	2026-06-07 01:09:21	10	1	1	2026-06-06 08:00:00	1

图 14 合法插入成功的演示截图

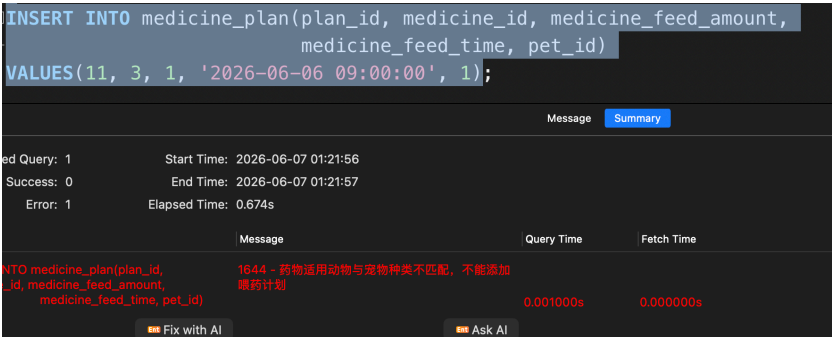


图 15 非法插入失败并报错的演示截图

6 存储过程控制下的更新操作（18 分）

6.1 评分点对应

本节要求说明功能、存储过程功能、涉及的关系表、连接字段、更改字段与规则，并给出关键代码和程序演示。为体现工作量，本节选取三个代表场景：正常扣减、指定批次不存在，以及库存不足。

6.2 选取的代表场景

本系统把“执行喂药计划”作为存储过程控制下的更新操作。应用层通过执行计划接口调用数据库侧过程 `proc_execute_medicine_plan`，执行时系统会根据计划中的喂药量扣减对应药品批次的库存数量。

6.3 功能描述（1 分）

该操作所要完成的功能是：根据指定的喂药计划和药品批次，检查库存是否充足；如果条件满足，则扣减 `medicine_store` 表中对应批次的剩余库存。

6.4 存储过程功能描述（1 分）

存储过程 `proc_execute_medicine_plan` 在数据库内部完成四件事：查找计划对应的药物编号和喂药量，校验计划是否属于当前主人，锁定要扣减的药物批次记录，检查剩余量是否足够，最后更新库存并提交事务。

6.5 涉及的关系表、连接字段与更改规则（5 分）

涉及的关系表：`medicine_plan`、`pet`、`medicine_store`
表连接涉及字段：

1. `medicine_plan.pet_id = pet.pet_id`
2. `medicine_plan.medicine_id = medicine_store.medicine_id`
3. `pet.owner_id = medicine_store.owner_id`

更改字段及规则：

1. `medicine_store.medicine_store_remaining_amount` 需要被更新。
2. 更新规则为“当前剩余量减去计划喂药量”。
3. 只有在库存批次存在且剩余量大于等于喂药量时才允许执行更新。

6.6 代表事件 A：自动选择最早过期批次并扣减库存

当调用接口时，如果前端没有显式传入批次号，后端会先查询该计划所需药物的所有可用批次，按过期时间和批次号排序后选择最早可用的一条，再调用存储过程完成扣减。这说明应用层和数据库层并不是简单重复，而是各自承担了“选批次”和“做更新”的不同职责。

6.7 代表事件 B：指定批次不存在

如果用户传入了一个不存在的 `medicine_batch_no`，存储过程在读取 `medicine_store` 时得不到结果，就会抛出“未找到对应药物库存批次”的错误，并通过异常处理器回滚事务。

6.8 代表事件 C：库存不足

即使批次存在，如果该批次剩余数量小于计划中的喂药量，存储过程同样会终止执行并报出“药物库存不足，无法执行喂药计划”。这样可以避免出现负库存。

6.9 关键代码与程序演示（11 分）

存储过程的关键更新语句如下：

```

1 UPDATE medicine_store
2 SET medicine_store_remaining_amount =
3     medicine_store_remaining_amount - v_feed_amount
4 WHERE owner_id = p_owner_id
5     AND medicine_id = v_medicine_id
6     AND medicine_batch_no = p_medicine_batch_no;
```

Listing 4: 执行喂药计划存储过程中的关键更新语句

在数据库层，这一过程被放在 `start transaction` 与 `commit` 之间，并通过 `for update` 锁定目标库存行；在应用层，接口使用 `cursor.callproc(...)` 调用存储过程，并在异常时统一 `rollback`。

```

1 if not medicine_batch_no:
2     cursor.execute(
3         """
4         SELECT ms.medicine_batch_no
5         FROM medicine_store ms
6         JOIN medicine_plan mp
7             ON ms.medicine_id = mp.medicine_id
8         WHERE mp.plan_id = %s
9             AND ms.owner_id = %s
10            AND ms.medicine_store_remaining_amount >= mp.
11               medicine_feed_amount
12            ORDER BY ms.medicine_store_expire_time ASC, ms.
13               medicine_batch_no ASC
14            LIMIT 1
15        """,
16        (plan_id, owner_id),
17    )
18    batch_row = cursor.fetchone()
19    if not batch_row:
20        connection.rollback()
21        return json_response(message="没有可用药物批次可执行该计划",
22                             status=400)
23    medicine_batch_no = batch_row["medicine_batch_no"]
```

© 2006 The Authors
Journal compilation © 2006 Blackwell Publishing Ltd



图 18 库存扣减之前的演示截图



图 19 库存扣减成功的演示截图



图 20 库存不足时的演示截图

7 含有视图的查询操作（15 分）

7.1 评分点对应

本节要求说明查询功能、视图功能、涉及的关系表、连接字段、创建视图代码和查询代码，并给出演示过程。为了体现工作量，本节不只展示一个查询，而是选取三个具有代表性的视图查询场景。

7.2 视图总体设计

数据库中共定义了 4 个主要视图：

1. v_owner_pet_detail: 查看某个主人名下全部宠物；
2. v_food_store_detail: 查看食品库存及过期情况；

3. v_medicine_store_detail: 查看药品库存及过期情况;
4. v_pet_medicine_plan_detail: 联合查看宠物、主人、药品和库存信息。

这些视图把多表连接逻辑提前封装好,使应用层在查询时可以直接围绕业务对象写条件,而不用每次都重新拼接长 SQL。

7.3 代表查询 A: 查看某个主人的全部宠物

操作功能描述: 查询某个主人名下的全部宠物,并同时看到猫或狗对应的扩展属性。

视图功能描述: v_owner_pet_detail 把 owner_table、pet、cat、dog 连接到一起,适合页面展示宠物总览。

涉及的关系表: owner_table、pet、cat、dog

表连接字段:

1. owner_table.owner_id = pet.owner_id
2. pet.pet_id = cat.pet_id
3. pet.pet_id = dog.pet_id

7.4 代表查询 B: 查看药物库存及即将过期记录

操作功能描述: 查询某个主人当前的全部药品库存,并能进一步筛选已经过期或即将过期的记录。

视图功能描述: v_medicine_store_detail 连接主人、药品、药品厂商和药品库存,适合库存列表和过期提醒页面。

涉及的关系表: medicine_store、owner_table、medicine、medicine_manu

表连接字段:

1. medicine_store.owner_id = owner_table.owner_id
2. medicine_store.medicine_id = medicine.medicine_id
3. medicine.medicine_manu_id = medicine_manu.medicine_manu_id

7.5 代表查询 C: 查看某只宠物或某个主人的喂药计划

操作功能描述: 查询指定宠物或指定主人的全部喂药计划,并同时看到药物名称、宠物名称、总库存和最近过期时间。

视图功能描述: v_pet_medicine_plan_detail 把喂药计划、宠物、主人、药物和汇总后的库存信息放在一张视图中,是本系统信息密度最高的联合视图。

涉及的关系表: medicine_plan、pet、owner_table、medicine、medicine_store

表连接字段:

1. medicine_plan.pet_id = pet.pet_id
2. pet.owner_id = owner_table.owner_id
3. medicine_plan.medicine_id = medicine.medicine_id
4. 汇总子查询中 stock.owner_id = owner_table.owner_id
5. 汇总子查询中 stock.medicine_id = medicine.medicine_id

7.6 创建视图与后端查询代码（6 分）

以喂药计划视图为例，其代表性设计思路是：先连接 `medicine_plan`、`pet`、`owner_table` 和 `medicine`，再通过子查询聚合 `medicine_store` 得到总库存和最近过期时间。应用层则通过 `GET /me/pets`、`GET /me/medicine-store`、`GET /me/plans` 等接口，在视图基础上追加 `owner_id`、`pet_id`、`before_time` 等筛选条件。

```

1 CREATE VIEW v_pet_medicine_plan_detail AS
2 SELECT
3     mp.plan_id,
4     o.owner_id,
5     p.pet_id,
6     p.pet_name,
7     m.medicine_id,
8     m.medicine_name,
9     mp.medicine_feed_time,
10    mp.medicine_feed_amount,
11    stock.total_remaining_amount,
12    stock.nearest_expire_date
13 FROM medicine_plan mp
14 JOIN pet p
15     ON mp.pet_id = p.pet_id
16 JOIN owner_table o
17     ON p.owner_id = o.owner_id
18 JOIN medicine m
19     ON mp.medicine_id = m.medicine_id
20 LEFT JOIN (
21     SELECT owner_id, medicine_id,
22            SUM(medicine_store_remaining_amount) AS
23            total_remaining_amount,
24            MIN(medicine_store_expire_time) AS nearest_expire_date
25     FROM medicine_store
26     GROUP BY owner_id, medicine_id
27 ) stock
28     ON stock.owner_id = o.owner_id
29     AND stock.medicine_id = m.medicine_id;

```

Listing 7: 喂药计划联合视图的核心定义

```

1 sql = """
2     SELECT *
3     FROM v_pet_medicine_plan_detail
4     WHERE owner_id = %s
5 """
6 params = [owner_id]
7
8 if pet_id:
9     sql += " AND pet_id = %s"
10    params.append(pet_id)

```

```
11
12 if before_time:
13     sql += " AND medicine_feed_time <= %s"
14     params.append(before_time)
15
16 sql += " ORDER BY medicine_feed_time"
17 cursor.execute(sql, tuple(params))
18 rows = cursor.fetchall()
```

Listing 8: app.py 中基于视图的查询逻辑

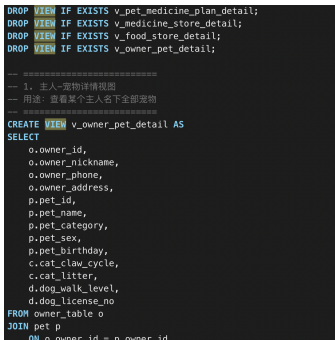


图 21 创建视图的 SQL 截图

7.7 程序演示 (4 分)

- 1. 登录后查看本人全部宠物。
- 2. 查看本人药品库存，并通过日期参数筛选即将过期的记录。
- 3. 查看本人全部喂药计划，并限定宠物与截止时间进行筛选。



图 23 视图查询结果演示截图一



图 24 视图查询结果演示截图二

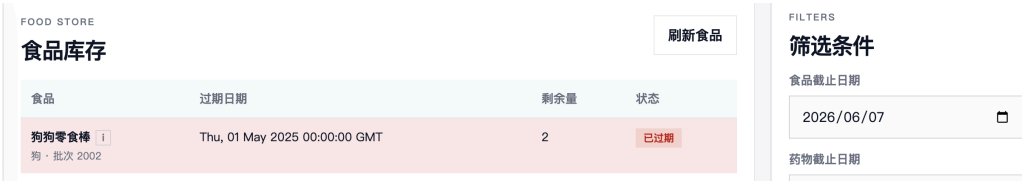


图 25 视图查询结果演示截图三



图 26 视图查询结果演示截图四



图 27 视图查询结果演示截图五

8 总结

本次作业围绕宠物信息管理这一数据库工程，分别实现并展示了事务删除、触发器添加、存储过程更新和视图查询四类数据库应用场景。我在本次实验过程中体会到了前后端如何协同工作，也熟练掌握了 sql 语句，总体来看收获颇丰。